

Closure Models, Physics-Informed Neural Networks, Neural Operators, and World Models

A unified numerical-approximation perspective

August 2026

Abstract

This chapter develops a common language for several ideas that are often discussed in separate communities: closure modeling in fluid dynamics and kinetic theory, physics-informed neural networks, neural operators, structure-preserving scientific machine learning, and world models. The central observation is deliberately simple. A neural network is a flexible numerical approximation family; the decisive question is *which mathematical object it is approximating*. It may approximate one solution of a differential equation, an input-to-solution operator, a constitutive or closure relation, a dynamical vector field, a time-advance map, an invariant manifold, or an action-conditioned transition kernel. Once that object is identified, the roles of equations, data, numerical solvers, physical constraints, and recursive prediction become much clearer.

Particular attention is given to three complementary research programs. Work by Zaher Hani and collaborators on long-time particle-to-kinetic, kinetic-to-fluid, and wave-to-kinetic limits supplies canonical physical settings in which closure, factorization, cumulants, and interaction history can be tested against a known asymptotic answer. Work by Andrew Christlieb and collaborators develops hyperbolic machine-learned kinetic moment closures, while Qi Tang and collaborators study stabilized neural ODEs, fast-slow neural networks, symplectic neural maps, energetic variational learning, and local-global neural operators. These methods are compared with classical residual PINNs and with neural operators in the standard function-space sense. The final sections explain how the same approximation viewpoint extends naturally to world models: learned internal simulators that estimate latent state, predict action-conditioned futures, and support planning or policy learning.

Contents

1	The organizing idea: what mathematical object is the network approximating?	6
2	Closure models and closure theories in fluid dynamics	7
2.1	Closure begins before turbulence	7
2.2	The general closure mechanism	7
2.3	Reynolds-averaged turbulence and the moment hierarchy	8

2.4	RANS closures	8
2.4.1	Linear eddy viscosity	9
2.4.2	Nonlinear algebraic stress and Reynolds-stress transport	9
2.5	Large-eddy simulation closure	9
2.5.1	Functional closures	10
2.5.2	Structural closures and implicit LES	10
2.6	Statistical closure theories	10
2.7	Kinetic and moment closure	10
2.8	Mori–Zwanzig: exact closure generally contains memory	11
2.9	Data-driven closure	11
2.10	What makes a closure credible?	12
3	Zaher Hani and collaborators: rigorous scale limits as canonical closure benchmarks	12
3.1	What the three results contribute	13
3.1.1	Hard spheres to Boltzmann	13
3.1.2	Boltzmann to fluid	13
3.1.3	Nonlinear waves to wave kinetics	13
3.2	Why these are unusually good benchmark settings	14
3.3	From proof objects to a computational evaluation harness	15
3.4	What transfers to TEMPEST—and what does not	15
4	Neural networks as numerical approximation families	15
4.1	The finite-element analogy	16
4.2	Approximating one function	16
4.3	Approximating a map between functions	16
4.4	Approximating a missing term	17
4.5	Approximating a vector field or flow map	17
4.6	Approximating geometry or latent state	17
5	Classical physics-informed neural networks	17
5.1	The narrow, original meaning	18
5.2	Forward and inverse problems	18
5.3	Where the physics enters	18

5.4	Why PINNs can be difficult	19
6	Neural operators	19
6.1	From one solution to a family of solutions	19
6.2	A standard neural operator does not require knowing a PDE	20
6.3	Fourier neural operators and discretization	20
6.4	PINO: the bridge between PINNs and neural operators	20
6.5	Neural operators are not automatically physical	21
7	Soft physics and hard physics	21
7.1	Soft physics	21
7.2	Hard physics	21
8	Andrew Christlieb’s program: learned kinetic closures inside admissible PDE models	22
8.1	The kinetic moment problem	22
8.2	Radiative-transfer closure I: learn the gradient relation	23
8.3	Radiative-transfer closure II: global hyperbolicity	23
8.4	Radiative-transfer closure III: characteristic-speed parameterizations	23
8.5	Nonlinear BGK moment closure	24
8.6	Learning a collision operator from molecular dynamics	24
8.7	Why this is not a classical PINN	25
8.8	Why this is not ordinarily called a neural operator	25
8.9	The strengths and limits of the program	26
9	Qi Tang’s program: structure-preserving learned dynamics, reduction, and operators	26
9.1	Stabilized neural ODEs	26
9.2	Fast-slow neural networks	27
9.3	Symplectic Hénon networks and Poincaré maps	28
9.4	Energetic variational learning	28
9.5	Local-global neural operator for conservation laws	28
9.6	A classification of Tang’s projects	29
9.7	The common methodological pattern	29

10 Side-by-side comparison: PINNs, neural operators, closures, and structured dynamics	30
10.1 The computational economics	31
10.2 Which approach is natural for which task?	31
10.2.1 PINNs	31
10.2.2 Neural operators	31
10.2.3 Learned closures	32
10.2.4 Structure-preserving learned dynamics	32
11 Do these methods require a PDE?	32
11.1 Three distinct ways a PDE can enter	33
11.2 No PDE does not mean no model assumption	33
12 The high-level numerical-approximation view	34
12.1 A PINN approximates a function obeying an equation	34
12.2 A neural operator approximates an input-output map	34
12.3 A learned closure approximates a missing relation	34
12.4 A neural ODE approximates a differential law	34
12.5 A learned flow map approximates finite-time evolution	34
12.6 A structure-preserving network approximates within a restricted class	34
13 World models	35
13.1 The internal-simulator viewpoint	35
13.2 The state is usually hidden	35
13.3 What makes it a world model?	36
13.4 Planning with a world model	36
13.5 A world model need not reconstruct everything	36
13.6 Generative interactive environments	37
13.7 A neural operator can function as a world model	37
13.8 World models and closure models	37
13.9 The controlled transition-operator viewpoint	38
13.10 Why world modeling is harder than ordinary supervised approximation	38
13.10.1 Compounding error	38
13.10.2 Distribution shift under planning	39

13.10.3 Hidden state and memory	39
13.10.4 Multiple possible futures	39
13.10.5 Task relevance versus physical fidelity	39
13.11 The grossest possible world-model summary	39
14 A single conceptual ladder	40
15 Practical questions for reading a scientific-ML paper	40
15.1 What is the target object?	40
15.2 Where does the supervision come from?	41
15.3 What structure is hard, and what structure is soft?	41
15.4 What remains of the conventional solver?	41
15.5 What is the relevant notion of generalization?	41
15.6 Is the model tested in closed loop?	41
15.7 What failure is ruled out mathematically?	41
16 Final synthesis	42
A Compact glossary	43
B One-page comparison table	44

1 The organizing idea: what mathematical object is the network approximating?

At an extremely high level, a neural network is a flexible, parameterized family of functions. Training chooses parameters θ so that one member of that family approximates a desired mathematical object. The object may be a scalar-valued function, a vector field, an operator between function spaces, a constitutive relation, a coordinate transformation, or a probability kernel.

The vocabulary changes from field to field because the *role* of the approximation changes. The same broad mathematical technology can therefore appear under very different names.

The gross-level unification

neural network = a flexible numerical ansatz for a function, map, operator, or kernel.

The scientifically meaningful questions are:

1. What object is being approximated?
2. What information is used to fit it: equations, data, or both?
3. Which mathematical properties are merely encouraged, and which are guaranteed by construction?
4. Does the network replace the whole numerical solver, or only one component of it?
5. Will the learned object be evaluated once, integrated in time, or composed recursively for long rollouts?

A useful first taxonomy is

$$\begin{array}{ll}
 u_\theta(x, t) \approx u(x, t), & \text{a particular solution,} \\
 \mathcal{G}_\theta(a) \approx \mathcal{G}^\dagger(a), & \text{an input-to-solution operator,} \\
 \mathcal{C}_\theta(m, \nabla m) \approx \text{an unresolved closure term,} & \\
 F_\theta(z) \approx F(z), & \text{a dynamical vector field,} \\
 \Phi_\theta(z, a) \approx \Phi(z, a), & \text{a controlled time-advance map,} \\
 h_\theta(x) \approx h(x), & \text{an invariant or slow manifold,} \\
 P_\theta(dz' \mid z, a) \approx P(dz' \mid z, a), & \text{a stochastic transition kernel.}
 \end{array} \tag{1}$$

This classification already separates the major paradigms:

Paradigm	Typical learned object	Numerical role
Classical PINN	One function $u_\theta(x, t)$	Neural trial function for one forward or inverse problem
Neural operator	Map $a(\cdot) \mapsto u(\cdot)$	Amortized solver for a family of problem instances
Learned closure	Missing relation $\mathcal{C}_\theta$	Component inserted into a retained reduced PDE or ODE model
Neural ODE	Vector field F_θ	Learned right-hand side integrated by an ODE solver
Learned flow map	Step map Φ_θ	Direct state-to-state surrogate, often recursively composed
World model	Latent state model and transition kernel	Internal simulator for prediction, counterfactual rollout, and planning

The rest of the chapter makes these distinctions precise, beginning with the classical closure problem in fluid dynamics.

2 Closure models and closure theories in fluid dynamics

2.1 Closure begins before turbulence

Fluid mechanics is built from balance laws. Conservation of mass, momentum, and energy constrains the evolution of density, velocity, and internal energy, but these balances do not by themselves specify the material response. A momentum equation contains a stress tensor; an energy equation contains a heat flux. To obtain a usable continuum model one introduces constitutive relations, such as

$$\sigma = -pI + 2\mu S + \lambda(\nabla \cdot u)I, \quad q = -\kappa \nabla T,$$

together with an equation of state. The Newtonian stress law, Fourier heat conduction, and the equation of state are already *closures*. They replace microscopic information by relations among retained macroscopic variables.

Thus the Navier–Stokes–Fourier equations are not raw conservation laws. They are a closed continuum model of a more microscopic kinetic system.

2.2 The general closure mechanism

Suppose a full state $X(t)$ evolves by

$$\dot{X} = \mathcal{F}(X),$$

but we retain only reduced variables

$$a = \Pi X.$$

Projecting or averaging the full equations often gives an exact identity for a , but the identity contains quantities depending on the discarded part of X . A closure replaces those unresolved quantities by

a model involving the retained variables, perhaps including gradients, history, stochastic forcing, or auxiliary variables.

Exact reduced equation versus approximate closure

Averaging, filtering, taking moments, or projecting modes can produce an exact reduced equation. The approximation enters when unresolved correlations, fluxes, memory terms, or conditional expectations are replaced by a tractable model.

This distinction is essential. The reduced equation and the closure are conceptually different objects, even when they are written together as one model.

2.3 Reynolds-averaged turbulence and the moment hierarchy

For incompressible Navier–Stokes,

$$\partial_t u_i + u_j \partial_j u_i = -\partial_i p + \nu \Delta u_i, \quad \partial_i u_i = 0, \quad (2)$$

write

$$u_i = U_i + u'_i, \quad \langle u'_i \rangle = 0.$$

Averaging gives

$$\partial_t U_i + U_j \partial_j U_i = -\partial_i \bar{p} + \nu \Delta U_i - \partial_j R_{ij}, \quad (3)$$

where

$$R_{ij} = \langle u'_i u'_j \rangle$$

is the Reynolds-stress tensor. The equation for the mean velocity is not closed because R is not determined by U .

One can derive a transport equation for R_{ij} . Schematically,

$$\frac{DR_{ij}}{Dt} = P_{ij} + \Phi_{ij} - \varepsilon_{ij} + D_{ij}, \quad (4)$$

where production is largely determined by resolved mean gradients, while pressure–strain redistribution, dissipation, and turbulent transport involve new correlations. Third moments such as

$$\langle u'_i u'_j u'_k \rangle$$

appear; their equations involve fourth moments; and so on. This is the turbulence moment hierarchy.

2.4 RANS closures

Reynolds-averaged Navier–Stokes models seek mean quantities without resolving individual turbulent eddies (Pope, 2000).

2.4.1 Linear eddy viscosity

The classical Boussinesq form is

$$R_{ij} = \frac{2}{3}k\delta_{ij} - 2\nu_t S_{ij}, \quad k = \frac{1}{2}R_{ii}, \quad (5)$$

with

$$S_{ij} = \frac{1}{2}(\partial_j U_i + \partial_i U_j).$$

The eddy viscosity ν_t models turbulent momentum transport. The key restriction is tensorial: the anisotropic stress is assumed locally aligned with the mean strain.

A mixing-length closure may use

$$\nu_t = \ell_m^2 \left| \frac{dU}{dy} \right|.$$

One-equation models transport a viscosity-like or energy-like variable. Two-equation models transport two turbulent quantities, for example

$$\nu_t = C_\mu \frac{k^2}{\varepsilon} \quad \text{or} \quad \nu_t \sim \frac{k}{\omega}.$$

The extra equations do not eliminate closure; their production, diffusion, dissipation, and wall behavior must themselves be modeled.

2.4.2 Nonlinear algebraic stress and Reynolds-stress transport

A nonlinear eddy-viscosity model permits a richer tensor relation

$$b = \frac{R}{2k} - \frac{1}{3}I = \sum_n g_n(I_1, I_2, \dots) T^{(n)}(S, \Omega), \quad (6)$$

where $T^{(n)}$ are invariant tensor-basis elements formed from strain S and rotation Ω .

A Reynolds-stress transport model instead evolves the six independent entries of R , plus one or more scale variables. This gives more freedom for anisotropy, rotation, curvature, and secondary flows, but the pressure–strain, dissipation, and triple-correlation transport terms remain unclosed. Closure has moved deeper into the hierarchy rather than disappeared.

2.5 Large-eddy simulation closure

Large-eddy simulation filters the velocity rather than taking a long-time or ensemble mean. Filtering gives

$$\partial_t \bar{u}_i + \bar{u}_j \partial_j \bar{u}_i = -\partial_i \bar{p} + \nu \Delta \bar{u}_i - \partial_j \tau_{ij}, \quad (7)$$

where

$$\tau_{ij} = \overline{u_i u_j} - \bar{u}_i \bar{u}_j \quad (8)$$

is the subgrid-scale stress. LES resolves large eddies and models the transfer across a chosen filter or grid scale.

2.5.1 Functional closures

A functional model emphasizes the net transfer of energy between resolved and unresolved scales. The Smagorinsky model uses

$$\tau_{ij}^d = -2\nu_{\text{sgs}}\bar{S}_{ij}, \quad \nu_{\text{sgs}} = (C_s\Delta)^2|\bar{S}|, \quad (9)$$

following the subgrid-viscosity idea of Smagorinsky (1963). Its modeled energy transfer is

$$\Pi = -\tau_{ij}\bar{S}_{ij} = 2\nu_{\text{sgs}}\bar{S}_{ij}\bar{S}_{ij} \geq 0, \quad (10)$$

so the model always removes energy from the resolved field. This is stabilizing and represents the average forward cascade, but it excludes local backscatter.

Dynamic models infer coefficients from the resolved flow by comparing two filter levels, using the Germano identity (Germano et al., 1991).

2.5.2 Structural closures and implicit LES

Structural models seek to approximate the tensor τ itself. Scale-similarity models, gradient expansions, approximate deconvolution, and mixed models fall into this category. They can represent stress structure and backscatter more faithfully, but they may supply too little net dissipation for a stable computation.

In implicit LES no explicit subgrid stress is written. Dissipation from reconstruction, limiting, fluxes, and time stepping acts as an effective closure. The numerical discretization is then part of the physical model rather than a neutral implementation detail.

2.6 Statistical closure theories

In homogeneous turbulence one can derive evolution equations for two-point correlations or spectra. A two-point covariance

$$C_{ij}(x - y, t) = \langle u_i(x, t)u_j(y, t) \rangle$$

or an energy spectrum $E(k, t)$ evolves through third-order correlations representing triadic interactions. Equations for third-order correlations involve fourth-order correlations.

Classical closure strategies include quasi-normal approximations, eddy-damped quasi-normal models, direct-interaction approximation, and Lagrangian-history variants. Kraichnan's direct-interaction program introduced response functions alongside correlations and derived self-consistent spectral equations (Kraichnan, 1959).

An essential requirement is *realizability*. A modeled covariance must remain positive semidefinite, and an energy spectrum cannot become negative. A statistical closure that violates these conditions is not merely inaccurate; it no longer corresponds to any probability law.

2.7 Kinetic and moment closure

Let $f(x, v, t)$ be a kinetic distribution. Its moments are

$$M^{(n)}(x, t) = \int v^{\otimes n} f(x, v, t) dv. \quad (11)$$

The equation for the n -th moment generally contains the $(n + 1)$ -st moment. Retaining density, momentum, and energy does not automatically determine stress and heat flux.

Different assumptions on f produce different fluid closures:

- a local Maxwellian gives compressible Euler;
- near-equilibrium asymptotics give Navier–Stokes–Fourier;
- Grad systems retain finitely many polynomial or Hermite moments;
- Gaussian closures retain anisotropic covariance information;
- maximum-entropy closures choose a distribution by constrained entropy optimization.

Entropy-based moment hierarchies make structural goals such as hyperbolicity, realizability, and entropy dissipation explicit (Levermore, 1996).

2.8 Mori–Zwanzig: exact closure generally contains memory

Projection-operator theory gives a particularly clean abstract account. If the full dynamics contain resolved variables a and unresolved variables b , then eliminating b leads formally to a generalized Langevin equation

$$\dot{a}(t) = \mathcal{M}(a(t)) + \int_0^t \mathcal{K}(a(t-s), s) ds + \eta(t). \quad (12)$$

The three pieces are an instantaneous term, a memory term, and an orthogonal-dynamics or fluctuating term (Mori, 1965; Zwanzig, 1961).

A crucial closure lesson

Exact reduced dynamics are generally nonlocal in time and depend on unresolved initial information. A local deterministic closure

$$\dot{a} = F(a)$$

is therefore a modeling assumption, not a generic consequence of eliminating variables.

This point will return later. A neural ODE trained on coarse observations may not mention the original PDE, yet it is implicitly making a Markovian closure assumption.

2.9 Data-driven closure

Machine learning changes the representation or calibration of the closure; it does not remove the closure problem. One may learn

$$R_{ij} = \mathcal{C}_{ij}(U, \nabla U, k, \varepsilon, \dots),$$

a subgrid stress, a correction to a production term, an unresolved kinetic moment, or a memory kernel.

A credible learned closure should respect as much known structure as possible:

$$\begin{aligned} &\text{frame invariance, tensor symmetry, conservation, realizability,} \\ &\text{correct asymptotic limits, stable energy behavior.} \end{aligned} \tag{13}$$

It is also important to distinguish two tests:

- a priori*: Does the closure match high-fidelity targets on stored data?
- a posteriori*: Does the coupled reduced equation plus closure produce the correct evolution?

A highly accurate offline stress fit can still destabilize the closed PDE, alter its attractor, or feed itself inputs outside the training distribution.

2.10 What makes a closure credible?

Across RANS, LES, kinetic moments, and data-driven reduction, several requirements recur.

1. **Symmetry and invariance.** The model should transform correctly under rotations, translations, and changes of inertial frame.
2. **Realizability.** For example, a Reynolds stress satisfies

$$R = \langle u' u'^T \rangle \succeq 0, \quad |R_{ij}|^2 \leq R_{ii} R_{jj}.$$

3. **Correct energy or entropy behavior.** The closure must put energy in the right scales and must not generate forbidden growth.
4. **Correct limits.** It should recover laminar, near-equilibrium, diffusive, near-wall, or mesh-refined limits where appropriate.
5. **Nonlocality and memory.** A purely local instantaneous closure is a substantive simplification.
6. **Well-posedness.** A plausible stress formula is not enough if the resulting PDE loses hyperbolicity, parabolicity, or numerical stability.

The next sections explain how neural approximations enter these problems, and why different scientific-machine-learning paradigms learn different mathematical objects.

3 Zaher Hani and collaborators: rigorous scale limits as canonical closure benchmarks

The work of Zaher Hani and collaborators is not a catalog of plug-in closure formulas. It is more foundational: it proves, in carefully controlled physical regimes, when a microscopic or wave description reduces to a kinetic equation and when a kinetic equation reduces further to fluid mechanics. Those results identify the scaling limit, the information discarded by the reduction, the time horizon over which the reduction remains valid, and the mathematical structures used to control the discarded information. For a computational closure program, that combination is unusually valuable.

The three relevant papers form a connected chain. Deng, Hani, and Ma derive the Boltzmann equation from a rarefied hard-sphere gas for arbitrarily long times within the lifespan of the Boltzmann solution (Deng et al., 2025a). A companion paper connects hard-sphere dynamics, through Boltzmann kinetic theory, to compressible Euler and incompressible Navier–Stokes–Fourier limits (Deng et al., 2025b). Deng and Hani obtain a long-time justification of the homogeneous wave kinetic equation over its full lifespan in a weakly nonlinear random-wave regime (Deng and Hani, 2024).

3.1 What the three results contribute

3.1.1 Hard spheres to Boltzmann

The microscopic system consists of elastically colliding hard particles in a rarefied-gas scaling. The reduced description is the one-particle distribution governed by the Boltzmann equation. The central closure assumption is propagation of chaos: finite collections of particles become approximately independent in the kinetic limit, so that the collision term can be written using products of one-particle distributions.

At finite density and finite scale separation, independence is not exact. Recollisions and shared collision ancestry produce connected correlations. The long-time proof propagates a structured cumulant ansatz, restarts partial expansions across time layers, and organizes collision histories through diagrams called *molecules*. Computationally, these objects suggest measurable diagnostics: projected two- and three-particle cumulants, recollision counts, collision-graph connectivity, factorization error, and the residual of the Boltzmann equation.

3.1.2 Boltzmann to fluid

The companion result continues the reduction from kinetic theory to classical fluid equations. Compressible Euler arises from a local-equilibrium regime, while incompressible Navier–Stokes–Fourier arises under a different hydrodynamic scaling and preparation of the kinetic data. This makes the work especially useful for closure research because it displays two successive information losses:

Newtonian particles $\longrightarrow$ Boltzmann distribution $\longrightarrow$ fluid fields and constitutive fluxes.

The first step suppresses particle correlations; the second suppresses velocity-space information beyond a few hydrodynamic moments. A numerical benchmark can therefore test not only whether the end-to-end fluid limit is recovered, but also which error enters at each reduction and whether a learned correction is attached to the correct level of the hierarchy.

3.1.3 Nonlinear waves to wave kinetics

The wave-turbulence result treats a different microscopic mechanism but the same closure logic. A nonlinear dispersive wave field has interacting Fourier modes; the reduced wave kinetic equation evolves a spectrum rather than the phases and higher joint statistics of those modes. Random-phase or near-Gaussian factorization is the analogue of molecular chaos. Connected spectral cumulants and resonant interaction histories measure the information discarded by that closure.

The long-time analysis again uses time layering and structured interaction diagrams, here called canonical layered gardens. This supplies a second canonical testbed in which the unresolved information is spectral rather than particle based. Agreement of a closure diagnostic across the particle and wave tracks would therefore be stronger evidence than success on two variants of the same simulator.

3.2 Why these are unusually good benchmark settings

These regimes are “standard” in a precise sense: hard-sphere kinetic theory, Boltzmann hydrodynamic limits, and weakly nonlinear wave kinetics are canonical models with explicit microscopic dynamics, explicit reduced equations, and named small parameters. They are substantially cleaner than a driven, bounded, magnetized, multispecies TEMPEST application, but that simplicity is an advantage at the validation stage.

Canonical reduction	Classical closure	Discarded information	Benchmark evidence
Hard particles $\rightarrow$ Boltzmann	Molecular chaos / factorization	Connected particle cumulants and collision ancestry	Convergence of marginals, collision statistics, cumulants, and Boltzmann residuals
Boltzmann $\rightarrow$ fluid	Local equilibrium and constitutive asymptotics	Non-Maxwellian velocity structure, higher moments, and kinetic remainder	Recovery of Euler or Navier–Stokes–Fourier fields, stresses, heat fluxes, and scaling rates
Nonlinear waves $\rightarrow$ wave kinetics	Random-phase or near-Gaussian spectral factorization	Spectral cumulants, phase information, and resonant interaction history	Convergence of spectra, transfer rates, spectral cumulants, and wave-kinetic residuals

They provide six properties that an evaluation harness needs:

1. **A known target.** The limiting kinetic or fluid equation is specified before any model is trained.
2. **A scaling coordinate.** Density, particle size, Knudsen number, wave nonlinearity, or scale separation can be swept toward and away from the trusted limit.
3. **Observable discarded information.** Cumulants, histories, non-equilibrium moments, and hierarchy residuals can be estimated from ensembles rather than discussed only qualitatively.
4. **A long-time test.** The results are designed to control kinetic behavior beyond the very short horizons on which a closure may look accurate accidentally.
5. **Controlled failure regimes.** One can deliberately leave the asymptotic assumptions by increasing density, strengthening correlations or coherent waves, reducing scale separation, or introducing steep kinetic structure.
6. **Error separation.** Numerical discretization, ensemble estimation, asymptotic-limit error, closure-model error, and learned-model error can be varied and reported separately.

3.3 From proof objects to a computational evaluation harness

For each canonical track, let U^ε denote the high-fidelity state at scale parameter ε , let $u^\varepsilon = \Pi U^\varepsilon$ be the retained state, and let $\mathcal{F}_0(u^\varepsilon)$ denote the declared classical reduced operator. The measurable closure defect is

$$\mathcal{D}^\varepsilon(t) = \partial_t u^\varepsilon(t) - \mathcal{F}_0(u^\varepsilon(t)). \quad (14)$$

The numerical program asks whether $\mathcal{D}^\varepsilon$ is predicted by a compact state built from projected cumulants, interaction-history summaries, non-equilibrium moments, or finite memory. Analytic, learned, and hybrid closures are then invoked through the same interface and scored with the same accuracy, stability, structure, reliability, and cost metrics.

The proof machinery is not itself assumed to be an efficient algorithm. Molecules, gardens, and cutting arguments first tell us what must be controlled. The computational question is whether low-order projections or learned compressions of those objects predict closure failure and improve the reduced solver. Negative answers are scientifically useful: they show that the proposed cumulant order, memory length, or representation is insufficient.

3.4 What transfers to TEMPEST—and what does not

The transferable contribution is the *experimental logic*: declare the hierarchy, identify the classical limit, expose the discarded correlations or moments, measure the exact closure defect, sweep a validity parameter, and test both asymptotic recovery and controlled breakdown. The common computational evaluation harness can carry that logic unchanged.

The physical equations and theorem assumptions do not transfer unchanged. TEMPEST applications may be magnetized, driven, bounded, multispecies, anisotropic, nonlocal, shock forming, or coupled across several physics packages. Their correlations need not be small, their memory may remain order one, and their natural reduced operator may be Braginskii, extended MHD, radiation hydrodynamics, or another application-specific model rather than Boltzmann or the homogeneous wave kinetic equation.

The correct role of Hani’s work in the bake-off

The canonical tracks validate whether the harness can detect, represent, and score closure in regimes where the reduction is mathematically understood. They do not claim that a hard-sphere or homogeneous-wave closure is an MTF closure. Each TEMPEST application must replace the equations, state, scaling parameters, constraints, observables, and coefficients while retaining the validated diagnostic and evaluation protocol.

4 Neural networks as numerical approximation families

4.1 The finite-element analogy

A traditional numerical method begins by choosing an approximation family. A finite-element method writes

$$u_h(x, t) = \sum_{j=1}^N c_j \phi_j(x, t), \quad (15)$$

where the basis functions ϕ_j are prescribed and the coefficients c_j are determined from a discrete variational or algebraic problem. A spectral method uses Fourier modes or orthogonal polynomials. A radial-basis method uses translated kernels.

A neural approximation instead chooses a nonlinear parameterized family

$$u_\theta(x, t) = W_L \sigma(W_{L-1} \sigma(\cdots \sigma(W_1[x, t] + b_1) \cdots) + b_{L-1}) + b_L. \quad (16)$$

The parameters θ consist of weights and biases. The neural network is therefore not mysterious at this level: it is a large adaptive trial family built from compositions of affine maps and nonlinearities.

The important novelty is not merely that this family can approximate a function u . The same technology can parameterize an entire solver map, an unknown constitutive law, a vector field, an invariant coordinate system, or a transition kernel.

Approximation technology	Typical representation
Finite differences	Values on a grid and local difference stencils
Finite elements	Piecewise-polynomial trial and test spaces
Spectral methods	Fourier, Chebyshev, or other global basis expansions
Radial basis methods	Sums of translated kernels
Neural methods	Compositions of affine maps and nonlinear activations, possibly with convolution, attention, integral kernels, recurrence, or geometric constraints

The choice of learned object determines the training problem and the deployment model.

4.2 Approximating one function

A coordinate network may represent

$$u_\theta : (x, t) \mapsto u_\theta(x, t).$$

This is the basic PINN viewpoint. The network is a trial function for one solution. The parameters are selected so that this trial function fits observations, initial and boundary conditions, and a differential-equation residual.

4.3 Approximating a map between functions

A neural operator represents

$$\mathcal{G}_\theta : a(\cdot) \mapsto u(\cdot).$$

The input may be an initial condition, forcing, coefficient field, boundary function, geometry encoding, or a collection of parameters. The output is a full function. The goal is not one solution but a reusable approximation to a whole solution operator.

4.4 Approximating a missing term

A learned closure represents

$$\mathcal{C}_\theta : \text{resolved state and features} \longmapsto \text{unresolved flux, stress, or moment}.$$

The known equations remain in place. A conventional numerical method solves them after the network supplies the missing relation.

4.5 Approximating a vector field or flow map

A neural ODE learns

$$\dot{z} = F_\theta(z),$$

and then an ODE solver integrates the learned vector field. A learned flow map instead predicts

$$z^{n+1} = \Phi_\theta(z^n),$$

thereby replacing one time step or a finite-time evolution map directly.

These are not interchangeable. Learning F asks for an infinitesimal generator; learning $\Phi_{\Delta t}$ asks for a finite-time map. Their stability, data requirements, and dependence on the chosen time increment differ.

4.6 Approximating geometry or latent state

A network can learn a coordinate transformation

$$z \longmapsto (z_{\text{slow}}, z_{\text{fast}}),$$

an invariant manifold

$$z_{\text{fast}} = h_\theta(z_{\text{slow}}),$$

or an encoder from observation history to latent state

$$z_t = E_\theta(o_0, a_0, \dots, o_t).$$

Here the network is not merely predicting the next physical field. It is constructing the state variables in which prediction or control becomes feasible.

5 Classical physics-informed neural networks

5.1 The narrow, original meaning

For a PDE

$$\mathcal{N}_\lambda[u] = f \quad \text{in } \Omega, \quad \mathcal{B}[u] = g \quad \text{on } \partial\Omega, \quad (17)$$

a classical PINN chooses a coordinate network $u_\theta(x, t)$ and minimizes a loss of the form

$$\begin{aligned} \mathcal{L}(\theta) = & \lambda_{\text{PDE}} \frac{1}{N_r} \sum_{j=1}^{N_r} |\mathcal{N}_\lambda[u_\theta](x_j, t_j) - f(x_j, t_j)|^2 \\ & + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}} + \lambda_{\text{IC}} \mathcal{L}_{\text{IC}} + \lambda_{\text{data}} \mathcal{L}_{\text{data}}. \end{aligned} \quad (18)$$

Automatic differentiation evaluates derivatives of u_θ . The network parameters are optimized so that one neural trial function approximately satisfies the equation, constraints, and data (Raissi et al., 2019).

The right mental model for a classical PINN

A classical PINN replaces a conventional discrete PDE solve by a nonlinear optimization problem over a neural trial space. It ordinarily learns *one solution* for one problem instance.

The same residual-based idea can be used for ODEs, differential-algebraic equations, kinetic equations, and some integro-differential or stochastic problems. Thus the phrase “PINN” need not literally require a partial differential equation. What it requires in its classical form is a known governing equation whose residual can be evaluated.

5.2 Forward and inverse problems

For a forward problem, the equation and coefficients are known and u is unknown. For an inverse problem, one may learn both u_θ and unknown parameters λ , for example

$$\partial_t u + \lambda u \partial_x u - \nu \partial_{xx} u = 0.$$

Sparse observations can inform λ and ν while the residual regularizes the inferred state.

This is one of the most natural uses of PINNs: the unknown state and unknown physical parameters are fitted jointly from incomplete observations under a known equation form.

5.3 Where the physics enters

In a vanilla PINN, the physics is primarily a *soft penalty*:

$$\mathcal{N}[u_\theta] \approx 0.$$

The optimizer may trade residual accuracy against initial conditions, boundary conditions, and data fit. A small sampled residual does not automatically imply

exact conservation,	positivity,	hyperbolicity,
entropy stability,	correct long-time dynamics.	

Many refined PINN methods add hard boundary parameterizations, weak or variational forms, domain decomposition, adaptive sampling, conservative formulations, causal training, and special architectures. Consequently, one should not reduce all PINNs to the simplest squared-residual implementation. Nevertheless, the defining learned object remains a solution function, and the governing equation is normally used directly in training.

5.4 Why PINNs can be difficult

The PDE residual, boundary residual, initial residual, and data residual can operate on different scales and generate conflicting gradients. High frequencies, sharp fronts, stiffness, chaotic dynamics, and long time intervals can make the optimization poorly conditioned.

There is also a distinction between an approximation error and an optimization error. The network family may be expressive enough to represent a good solution, yet gradient-based training may not find it. Classical numerical analysis often studies consistency and stability of a specified discretization. PINNs add a large nonconvex optimization layer between the approximation family and the computed answer.

6 Neural operators

6.1 From one solution to a family of solutions

Suppose a denotes problem data:

$$a = \{\text{coefficient field, initial condition, forcing, boundary data, geometry, parameters}\}.$$

A PDE or another process defines a map

$$\mathcal{G}^\dagger : a(\cdot) \mapsto u(\cdot). \quad (19)$$

A neural operator learns

$$\mathcal{G}_\theta \approx \mathcal{G}^\dagger$$

from many input-output pairs

$$\{a_j, u_j\}_{j=1}^N, \quad u_j = \mathcal{G}^\dagger(a_j).$$

DeepONet, Fourier neural operators, graph neural operators, kernel neural operators, wavelet constructions, and operator transformers are architectural families for representing maps between functions (Lu et al., 2021; Li et al., 2021a; Kovachki et al., 2023).

The right mental model for a neural operator

A neural operator pays a potentially large offline training cost to build an amortized solver for an entire distribution of problem instances. Once trained, a new input function can be mapped to an output function very quickly.

A standard supervised objective is

$$\min_{\theta} \frac{1}{N} \sum_{j=1}^N \|\mathcal{G}_\theta(a_j) - u_j\|_{\mathcal{U}}^2. \quad (20)$$

The network is shared across many instances, unlike a classical PINN that is normally reoptimized for one instance.

6.2 A standard neural operator does not require knowing a PDE

The operator-learning problem is mathematically

$$\text{input function} \mapsto \text{output function}.$$

Paired data are sufficient. The outputs could come from a PDE solver, laboratory measurements, molecular dynamics, a different simulator, or any reproducible function-to-function transformation. A PDE may have generated the data, but the training algorithm need not evaluate or even know it.

This yields an important distinction:

PDE as training constraint:	$\mathcal{N}[\mathcal{G}_\theta(a)]$ is evaluated;
PDE as data source:	only pairs (a, u) are used;
PDE as architectural inspiration:	conservation, locality, or multiscale structure shapes the model.

6.3 Fourier neural operators and discretization

An FNO alternates pointwise nonlinearities with global Fourier-domain integral operators. In simplified form,

$$v_{\ell+1}(x) = \sigma \left(W_\ell v_\ell(x) + \mathcal{F}^{-1}(R_\ell(k) \mathcal{F} v_\ell(k))(x) \right). \quad (21)$$

The operator viewpoint is intended to separate the learned map from one fixed grid, although actual performance still depends on sampling, truncation, architecture, and the input distribution.

The attractive computational economics are

$$\text{expensive data generation and training} \longrightarrow \text{very cheap repeated inference}.$$

This is particularly useful for uncertainty quantification, inverse design, optimization, data assimilation, control, and parameter sweeps requiring many related solves.

6.4 PINO: the bridge between PINNs and neural operators

A physics-informed neural operator combines operator regression with equation constraints:

$$\mathcal{L}_{\text{PINO}} = \mathcal{L}_{\text{operator data}} + \lambda_{\text{PDE}} \mathbb{E}_{a \sim \mu} \left[\|\mathcal{N}_a[\mathcal{G}_\theta(a)]\|^2 \right]. \quad (22)$$

It learns over many problem instances, but uses the PDE residual as supervision or regularization (Li et al., 2021b).

Thus:

	Learned object	Training scope
PINN	One solution u_θ	One instance, usually equation-constrained optimization
Neural operator	Operator $\mathcal{G}_\theta$	Many input-output instances, often supervised
PINO	Operator $\mathcal{G}_\theta$	Many instances with data and/or PDE constraints

6.5 Neural operators are not automatically physical

A neural operator can have low one-step or fieldwise error while violating conservation, positivity, causality, symmetry, or stability. Recursive rollouts may amplify small errors. The operator can also fail under distribution shift: a new coefficient field, initial condition, forcing, or geometry may lie outside the training measure.

The phrase “operator learning” therefore identifies the learned object. It does not by itself certify physical admissibility or long-time reliability.

7 Soft physics and hard physics

The contrast between residual PINNs and much structure-preserving scientific machine learning can be summarized as follows.

7.1 Soft physics

A loss term says

“Please make this violation small on sampled training states.”

For example, one might penalize nonreal characteristic speeds by

$$\mathcal{L}_{\text{hyp}} = \sum_q |\text{Im } \lambda_q(A_\theta)|^2. \quad (23)$$

The optimizer can trade this penalty against other objectives, and the condition is checked only where training samples or collocation points are supplied.

7.2 Hard physics

An architectural parameterization says

“Every model produced by this architecture belongs to an admissible class.”

Examples include

$$\begin{aligned} A_\theta(m) &\text{ is hyperbolic by construction,} \\ (D\Phi_\theta)^T J(D\Phi_\theta) &= J \text{ by composition of symplectic layers,} \\ \mathcal{M}_\theta &\text{ is an attracting invariant manifold by design,} \\ \sum_i (u_i^{n+1} - u_i^n) &= 0 \text{ through a conservative projection.} \end{aligned}$$

Hard structure does not guarantee predictive accuracy. A perfectly hyperbolic closure can be quantitatively wrong; a perfectly symplectic map can approximate the wrong dynamics. What it guarantees is that the learned model will not fail in that particular prohibited way.

The Christlieb–Tang methodological theme

Retain trusted mathematical structure, isolate the unknown or expensive component, parameterize only an admissible class for that component, and learn the remaining degrees of freedom from data.

8 Andrew Christlieb’s program: learned kinetic closures inside admissible PDE models

Christlieb’s work is the most direct continuation of the classical closure story. The network ordinarily does not represent the full kinetic solution and does not replace the entire reduced PDE solver. It supplies a carefully chosen missing relation in a moment system.

8.1 The kinetic moment problem

In one spatial and velocity dimension, let

$$m_k(x, t) = \int_{\mathbb{R}} v^k f(x, v, t) dv. \quad (24)$$

Taking moments of a transport or kinetic equation produces a hierarchy resembling

$$\partial_t m_k + \partial_x m_{k+1} = S_k, \quad k = 0, 1, \dots \quad (25)$$

If one retains $m_0, \dots, m_N$, the last equation contains the unknown m_{N+1} . A closure is needed.

A straightforward learned closure would predict

$$m_{N+1} = \mathcal{C}_\theta(m_0, \dots, m_N).$$

Christlieb and collaborators instead developed a gradient-based relation of the form

$$\partial_x m_{N+1} \approx \sum_{j=0}^N a_{j,\theta}(m_0, \dots, m_N) \partial_x m_j. \quad (26)$$

The network predicts the coefficients $a_{j,\theta}$, and the closed moment PDE is advanced by a numerical method.

What the neural network is approximating

In this program the network approximates neither $f(x, v, t)$ nor the full input-to-solution map. It approximates the missing high-moment flux contribution needed by the last retained moment equation.

This is a genuine constitutive closure:

known moment equations + learned top-order closure + conventional hyperbolic solver.

8.2 Radiative-transfer closure I: learn the gradient relation

The first paper in the radiative-transfer series proposes learning the gradient of the unclosed moment rather than learning the moment itself (Huang et al., 2021a). This choice is motivated by the exact free-streaming structure and by the role the missing derivative plays in the retained PDE.

The conceptual move is important. A generic regression problem would ask the network to approximate an available target. A closure-oriented design asks a more structural question:

Which unresolved quantity appears in the reduced equation, and what is the smallest learned relation needed to make that reduced equation executable?

The gradient closure can also be normalized in ways adapted to the moment variables. Numerical comparisons in the series show that predicting the derivative relation can outperform learning the high moment pointwise and can improve over conventional low-order moment closure in the tested transport regimes.

8.3 Radiative-transfer closure II: global hyperbolicity

A learned coefficient vector a_θ changes the principal matrix of the moment system. A generic network may produce a matrix with complex eigenvalues. The resulting PDE can lose hyperbolicity, so even a good offline regression fit may generate an ill-behaved time evolution.

The second paper seeks a symmetric positive-definite symmetrizer and derives restrictions under which the learned closure system is globally symmetrizable hyperbolic (Huang et al., 2021c). The construction is also arranged to retain dissipative structure and the correct diffusion limit as the Knudsen number tends to zero.

This is stronger than adding a loss term that penalizes complex eigenvalues on the training set. Hyperbolicity is encoded in the admissible parameterization of the closed PDE.

8.4 Radiative-transfer closure III: characteristic-speed parameterizations

A later construction exploits the lower-Hessenberg structure of the closure matrix. The network predicts spectral or polynomial information, and algebraic relations convert that information into closure coefficients (Huang et al., 2021b).

The work makes a useful tradeoff explicit:

- one architecture guarantees characteristic speeds in the physical interval but only weak hyperbolicity;
- another guarantees strict hyperbolicity but does not automatically bound all characteristic speeds by the physical speed limit.

This illustrates a general lesson in structure-preserving learning. Different desirable properties need not be simultaneously achievable through one simple parameterization. A mathematically serious model should state exactly which guarantee it provides.

8.5 Nonlinear BGK moment closure

The BGK kinetic model is nonlinear because the relaxation equilibrium depends on moments of f . Christlieb, Ding, Huang, and Krupansky extend the gradient-closure strategy to a Grad moment expansion of BGK (Christlieb et al., 2025).

The network is trained on moment data from kinetic solutions and predicts coefficients in a relation of the form (26). Its output layers are designed to enforce

$$\boxed{\text{hyperbolicity}} \quad \text{and} \quad \boxed{\text{Galilean invariance}}.$$

Galilean invariance means that a uniform shift of inertial frame does not change the physical content of the closure. A generic fully connected network acting on raw moments would not satisfy this automatically.

The model is coupled to a path-conservative numerical treatment because the highest retained moment equation is not conservative under the gradient closure. This is a revealing detail: the learned closure and the numerical PDE method must be designed together. One cannot assess the network independently of the closed system in which it will operate.

A schematic training loss is

$$\mathcal{L}(\theta) = \sum_{\text{samples}} \left| \partial_x m_{N+1} - \sum_{j=0}^N a_{j,\theta}(m) \partial_x m_j \right|^2. \quad (27)$$

The target is not a complete solution field. It is the unresolved differential relation.

8.6 Learning a collision operator from molecular dynamics

A later project moves further down the modeling hierarchy. At the particle level, molecular dynamics contains many-body interactions and correlations. At the kinetic level these effects are compressed into a collision operator

$$\partial_t f + v \cdot \nabla_x f = \mathcal{Q}(f). \quad (28)$$

Rather than selecting a classical Landau- or Boltzmann-type form and calibrating a few coefficients, the model learns a generalized anisotropic collision operator directly from molecular-dynamics data (Zhao et al., 2025).

The operator is placed in a generalized metriplectic form designed to preserve

$$\begin{aligned} \int \mathcal{Q}(f) \, dv &= 0, & \text{mass,} \\ \int v \mathcal{Q}(f) \, dv &= 0, & \text{momentum,} \\ \int |v|^2 \mathcal{Q}(f) \, dv &= 0, & \text{energy,} \end{aligned}$$

together with nonnegative entropy production. The learning procedure uses a weak formulation so that it need not estimate a high-dimensional probability density pointwise.

This is again closure modeling. The unknown object is larger than a moment relation, but it is still one component of a retained kinetic equation.

8.7 Why this is not a classical PINN

A classical PINN for BGK might represent

$$f_\theta(x, v, t) \approx f(x, v, t)$$

and minimize

$$\left\| \partial_t f_\theta + v \partial_x f_\theta - \frac{1}{\tau} (f_M[f_\theta] - f_\theta) \right\|^2. \quad (29)$$

Christlieb’s closure program instead retains the kinetic or moment modeling hierarchy and learns only the missing relation needed by a low-dimensional reduced system.

The difference is not cosmetic:

	Classical PINN	Christlieb-style closure
Learned object	Full solution f_θ or m_θ	Missing moment-gradient or collision relation
Equation use	Residual in training loss	Determines hierarchy, closure location, admissibility, and deployed PDE
Solver	Neural optimization produces solution	Conventional solver advances closed equations
Structure	Often soft unless specially designed	Hyperbolicity, invariance, or conservation built into closure class
New initial condition	Usually new optimization	Reuse closure and solve reduced PDE again

8.8 Why this is not ordinarily called a neural operator

Abstractly, every closure is an operator. But in current scientific-ML usage, a neural operator usually denotes a discretization-aware map between functions, such as

$$f_0(x, v) \longmapsto f(x, v, t)$$

or

$$m(x, 0) \mapsto m(x, t).$$

The Christlieb closure more typically learns a local or semilocal constitutive rule

$$(m, \partial_x m) \mapsto \partial_x m_{N+1}.$$

Its role is to complete a PDE, not to replace the entire PDE solution map.

8.9 The strengths and limits of the program

The strongest feature is structural control. The retained variables, missing term, characteristic matrix, invariances, and numerical method are identified explicitly. The model is modular and interpretable as a closure.

The price is specialization. One must know

1. which variables to retain;
2. where the hierarchy fails to close;
3. what functional form is plausible;
4. which conditions make the resulting PDE admissible;
5. how to solve the closed system robustly.

Hard constraints prevent specified pathologies but do not guarantee extrapolation accuracy. A hyperbolic model can remain well posed while being quantitatively wrong outside the training regime.

9 Qi Tang’s program: structure-preserving learned dynamics, reduction, and operators

Tang’s work occupies several categories. Some projects learn vector fields, some learn invariant manifolds, some learn symplectic flow maps, and some are genuine neural operators. The common thread is that dynamical or geometric structure is encoded in the model rather than left entirely to a regression loss.

9.1 Stabilized neural ODEs

A neural ODE learns a finite-dimensional vector field

$$\dot{u} = F_\theta(u).$$

In spatiotemporal systems, high-wavenumber behavior can be poorly captured by a generic dense network even when short-time state errors are small. Tang and collaborators propose a stabilized architecture

$$\dot{u} = L_\theta u + N_\theta(u), \tag{30}$$

where a sparse linear convolutional component and a dense nonlinear component are learned separately (Linot et al., 2023).

The split gives the model a more transparent mechanism for representing dissipative or high-frequency dynamics. Tests on viscous Burgers and Kuramoto–Sivashinsky emphasize not only one-step fit but shocks, high-wavenumber spectra, robustness to perturbed initial conditions, and whether long trajectories remain on the correct attractor.

This is best classified as data-driven system identification:

$$\boxed{\text{trajectory data} \mapsto \text{learned vector field} \mapsto \text{ODE integration.}}$$

The original PDE may have generated the training trajectories, but the learning method need not evaluate its residual.

9.2 Fast-slow neural networks

Consider a singularly perturbed system

$$\dot{x} = g(x, y, \varepsilon), \quad \varepsilon \dot{y} = f(x, y, \varepsilon), \quad 0 < \varepsilon \ll 1. \quad (31)$$

Under dissipative fast dynamics, trajectories approach a slow manifold

$$y = h(x, \varepsilon).$$

The reduced dynamics are approximately

$$\dot{x} = g(x, h(x, \varepsilon), \varepsilon).$$

The closure problem is to identify the manifold and the dynamics on it.

The fast-slow neural network learns an invertible transformation into fast-slow coordinates and a structured vector field in a modified Fenichel normal form (Serino et al., 2025). Schematically,

$$z \xrightarrow{H_\theta} (x, y), \quad \dot{y} = A_\theta(x, y, \varepsilon)y, \quad \dot{x} = \varepsilon g_\theta(x, y, \varepsilon), \quad (32)$$

with the fast dynamics parameterized so that the manifold $y = 0$ is invariant and attracting.

The architecture therefore imposes the existence of a trainable attracting slow manifold as a hard constraint. Restricting to that manifold yields a learned reduced model. The examples include a Grad moment system, two-scale Lorenz–96, and Abraham–Lorentz dynamics.

Closure by learned coordinates

A traditional closure chooses reduced variables first and then models their missing flux. A fast-slow network can instead learn coordinates in which the slow variables and the attracting closure manifold become explicit.

This places the method conceptually near inertial-manifold reduction, equation-free modeling, and Mori–Zwanzig ideas, rather than near a residual PINN.

9.3 Symplectic Hénon networks and Poincaré maps

Hamiltonian flow maps preserve a symplectic form. In canonical coordinates, a map Φ is symplectic when

$$(D\Phi)^T J (D\Phi) = J. \quad (33)$$

A generic neural map does not satisfy this identity, and small geometric violations can produce artificial damping, energy drift, or incorrect phase-space structure under repeated iteration.

Tang and collaborators use Hénon-network layers whose composition is exactly symplectic. In the toroidal magnetic-field application, the network learns a Poincaré map from observations of a conventional field-line-following algorithm (Burby et al., 2021). The architecture exactly preserves the relevant flux constraint while evaluating much faster than repeated field-line integration.

The network is learning

$$z_{n+1} = \Phi_\theta(z_n),$$

not a PDE solution. The primary input is map data, and the primary prior is symplecticity.

Later work constructs a symplectic gyroceptron for nearly periodic maps. The architecture is nearly periodic and symplectic and supports a discrete adiabatic invariant, improving long-time behavior by design (Duruiseaux et al., 2022).

Recent reduced-order Hamiltonian work combines a symplectic encoder-decoder with symplectic latent dynamics, so both dimension reduction and time evolution preserve the underlying geometric class (Chen et al., 2025).

One should distinguish symplecticity from exact conservation of the original Hamiltonian. A symplectic integrator or map preserves phase-space geometry and often exhibits excellent long-time energy behavior, but it need not keep the original Hamiltonian exactly constant at every step.

9.4 Energetic variational learning

Many dissipative systems possess an energy-dissipation law

$$\frac{d}{dt} \mathcal{E}[u] = -\mathcal{D}[u], \quad \mathcal{D}[u] \geq 0. \quad (34)$$

Tang and collaborators use energetic variational structure to infer unknown laws from probability-density or particle data (Lu et al., 2026). The governing information enters through an energy-dissipation principle rather than only through a pointwise strong-form PDE residual.

This is an instructive intermediate case. The method is physics-informed, but the “physics” is a variational law that organizes the admissible dynamics. It need not be presented as a standard PINN loss for the full PDE solution.

9.5 Local-global neural operator for conservation laws

Tang’s local-global neural operator is a genuine neural-operator project. It learns a one-step flow map

$$\mathcal{G}_{\Delta t} : u^n \mapsto u^{n+1} \quad (35)$$

for hyperbolic conservation laws (Wang et al., 2026).

The architecture combines

$$\mathcal{G}_\theta = \mathcal{G}_\theta^{\text{global}} + \mathcal{G}_\theta^{\text{local}}.$$

A Fourier branch models large-scale, long-range, relatively smooth interactions; a local multiresolution branch targets shocks, contacts, and localized nonsmooth features. The training objective combines physical-space prediction error with a high-frequency spectral term. As of August 2026, this work is a recent preprint and should be evaluated as such.

LGNO is much closer to FNO than to a PINN. It is supervised on high-order numerical solution data and learns a discrete state-transition operator. Its physics enters through the chosen flow-map target, local-global decomposition, conservation-oriented parameterization, boundary treatment, and spectral objective rather than primarily through a continuous PDE residual.

9.6 A classification of Tang’s projects

Project	Learned object	Hard or structural prior	Best classification
Stabilized neural ODE	Vector field $L_\theta u + N_\theta(u)$	Linear/nonlinear split adapted to scale behavior	Data-driven system identification
Fast-slow network	Coordinate map, full vector field, and slow manifold	Attracting invariant slow manifold	Dynamical closure and model reduction
HénonNet Poincaré map	Discrete flow or section map	Exact symplecticity/flux preservation	Structure-preserving map surrogate
Symplectic latent ROM	Encoder, decoder, and latent flow map	Symplectic full-to-latent geometry	Geometric reduced-order modeling
Energetic variational learning	Unknown potential, mobility, or dissipative law	Energy-dissipation formulation	Structure-aware model discovery
LGNO	Function-to-function time-step map	Local-global decomposition and conservation-law inductive bias	Neural operator

9.7 The common methodological pattern

Across these projects the workflow is approximately

1. identify the unknown, expensive, or unstable component;
2. derive the mathematical structure that a usable approximation should possess;
3. parameterize a neural class satisfying as much of that structure as possible;

4. learn the remaining degrees of freedom from high-fidelity data;
5. evaluate the result inside long-time dynamics, not only by offline regression error.

This is more specialized than writing down a universal residual loss, but it exposes failure modes and aligns the learned object with the eventual numerical computation.

10 Side-by-side comparison: PINNs, neural operators, closures, and structured dynamics

Feature	Classical PINN	Standard neural operator	Christlieb closure	Tang structured models
Learned object	One solution $u(x, t)$	Solution or transition operator	Missing flux, moment relation, or collision operator	Vector field, slow manifold, coordinate map, or flow map
Typical data	Collocation points, BC/IC, sparse observations	Many input-output function pairs	High-fidelity kinetic moment or particle data	Trajectories, map pairs, or numerical solution pairs
Equation required in training?	Usually yes	No, unless physics-informed	The known hierarchy is structurally essential	Varies; often no residual is evaluated
Conventional solver retained?	Usually not for the represented solution	Usually replaced at inference	Yes	Sometimes: ODE integration for vector fields; no for direct flow maps
Cross-instance reuse	Limited in classical form	Central purpose	Reuse closure across reduced solves	Reuse learned dynamics or map within training regime
Typical hard guarantee	Few unless specially added	Few unless specially added	Hyperbolicity, invariance, conservation, entropy structure	Symplecticity, attracting manifold, scale structure, or conservation
Main economy	Avoid dense solution data; useful for inverse problems	Amortize many forward solves	Replace only expensive unresolved physics	Stable or geometry-aware long-time surrogate

Feature	Classical PINN	Standard neural operator	Christlieb closure	Tang structured models
Main risk	Nonconvex optimization and residual imbalance	Distribution shift and rollout instability	Incorrect closure ansatz or extrapolation	Wrong structural class or accumulated dynamical error

10.1 The computational economics

A classical PINN solves, for each instance a ,

$$\theta^*(a) = \arg \min_{\theta} \mathcal{L}_{\text{PINN}}(\theta; a). \quad (36)$$

The online solve is itself a potentially expensive optimization.

A neural operator solves an expensive population-level training problem

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{a \sim \mu} \left\| \mathcal{G}_{\theta}(a) - \mathcal{G}^{\dagger}(a) \right\|^2, \quad (37)$$

then evaluates $\mathcal{G}_{\theta^*}(a)$ cheaply for new inputs drawn from a similar distribution.

A learned closure trains $\mathcal{C}_{\theta}$ offline, then still solves a reduced equation for each new instance:

$$\partial_t m + \nabla \cdot F(m, \mathcal{C}_{\theta}(m, \nabla m)) = S(m). \quad (38)$$

Inference is not one network call; it is a cheaper PDE solve than the original kinetic, molecular, or fully resolved model.

A learned vector field similarly requires numerical integration, while a learned flow map can be iterated directly. The latter is often faster but places more responsibility on the learned map's recursive stability.

10.2 Which approach is natural for which task?

10.2.1 PINNs

PINNs are especially natural when the main goal is an inverse problem: sparse or irregular observations, unknown physical coefficients, and a reasonably trusted governing equation. They can also be attractive when complete training-solution ensembles are unavailable.

10.2.2 Neural operators

Neural operators are natural when many related forward solves are required, a representative training ensemble can be generated, and very fast online evaluation matters. Typical settings include optimization, uncertainty quantification, control, and digital twins.

10.2.3 Learned closures

A closure approach is natural when the full model is known but expensive, a meaningful reduced model already exists, and the precise unresolved term can be isolated. It preserves a mature solver and often permits more direct enforcement of well-posedness and asymptotic structure.

10.2.4 Structure-preserving learned dynamics

A structured vector field or flow map is natural when long-time behavior is controlled by geometry, invariant manifolds, stiffness, or attractors, and when ordinary one-step regression produces unstable rollouts.

11 Do these methods require a PDE?

The answer is no. They all require assumptions and information, but a partial differential equation is only one possible source of structure.

Method	Must know a PDE?	What is actually required?
Classical PINN	Usually yes, more generally a governing equation	Residual that can be evaluated, plus initial/boundary constraints and possibly data
Standard neural operator	No	Paired input and output functions from simulations, experiments, or another process
PINO	Usually yes	Operator-learning setup plus a known equation used in the physics loss
Christlieb moment closure	Yes, structurally	Known kinetic/moment hierarchy, a specific closure location, and admissibility conditions
Stabilized neural ODE	No	Trajectory data and the assumption that a useful continuous-time vector field exists
Fast-slow network	No	A dynamical system with dissipative fast variables and an attracting slow manifold
Symplectic HénonNet	No	Map observations plus the assumption that the target map is symplectic or flux-preserving
Supervised PDE neural operator	Not during training	State-transition or input-solution pairs, often generated by a PDE solver
World model	No	Sequential observations, actions, and enough state or memory to predict decision-relevant futures

11.1 Three distinct ways a PDE can enter

The PDE is evaluated during training. For a PINN or PINO,

$$\mathcal{N}[u_\theta] \quad \text{or} \quad \mathcal{N}_a[\mathcal{G}_\theta(a)]$$

is explicitly computed. The equation is operationally required.

The PDE determines the architecture and deployed model. For a learned moment closure, the equation determines the moments, the missing term, the principal matrix, the hyperbolicity conditions, and the solver. Even if training uses only high-fidelity data, the PDE is structurally essential.

The PDE merely generated the data. A neural ODE or supervised neural operator can train only on arrays

$$u(t_0), u(t_1), \dots, u(t_N)$$

or pairs (a, u) . Experimental observations could replace simulation data without changing the core training formulation.

11.2 No PDE does not mean no model assumption

A neural ODE assumes the observed state is approximately Markovian in continuous time:

$$\dot{z} = F(z).$$

A one-step operator assumes

$$z_{n+1} = \Phi(z_n)$$

contains enough information for prediction. A fast-slow network assumes an attracting manifold. A symplectic network assumes the relevant map lies in a symplectic class. A world model assumes its latent state summarizes the history relevant to future consequences.

If coarse variables omit important hidden state, exact dynamics may require

$$\dot{z}(t) = F(z(t)) + \int_0^t K(z(t-s), s) ds + \eta(t). \quad (39)$$

A Markovian neural model is then itself a closure approximation, even if it never mentions the original PDE.

Better question

Instead of asking only, “Does this method use a PDE?”, ask:

What information about the governing dynamics is supplied to the learner: an equation, solution data, a closure location, a variational law, a geometric class, trajectories, actions, or observation history?

12 The high-level numerical-approximation view

The preceding discussion can be condensed without losing its conceptual content.

12.1 A PINN approximates a function obeying an equation

$$(x, t) \mapsto u_\theta(x, t), \quad \mathcal{N}[u_\theta] \approx 0.$$

The network is a nonlinear trial function analogous in role, though not in training, to a finite-element or spectral ansatz.

12.2 A neural operator approximates an input-output map

$$a(\cdot) \mapsto \mathcal{G}_\theta(a)(\cdot).$$

The network approximates the behavior of a solver over a family of inputs.

12.3 A learned closure approximates a missing relation

$$\text{resolved variables} \mapsto \text{unresolved stress, flux, moment, or collision effect.}$$

The learned object is inserted into retained equations.

12.4 A neural ODE approximates a differential law

$$z \mapsto F_\theta(z), \quad \dot{z} = F_\theta(z).$$

A numerical integrator then constructs trajectories.

12.5 A learned flow map approximates finite-time evolution

$$z_t \mapsto z_{t+\Delta t}.$$

The map is recursively composed. Stability of composition is central.

12.6 A structure-preserving network approximates within a restricted class

$$\Phi_\theta \in \{\text{symplectic maps}\}, \quad A_\theta \in \{\text{hyperbolic systems}\}, \quad \mathcal{M}_\theta \in \{\text{attracting manifolds}\}.$$

The approximation family is narrowed so that every candidate respects a chosen mathematical property.

The simplest accurate summary

The neural network is playing the role of a numerical approximation: sometimes of a function that obeys an equation, sometimes of a map that takes input to output, sometimes of a missing term inside an equation, and sometimes of the coordinates or geometry in which the dynamics

become simple.

13 World models

13.1 The internal-simulator viewpoint

A world model extends the same approximation idea to an agent interacting with an environment. At its simplest, the physical or abstract state x_t evolves under an action a_t :

$$x_{t+1} = F(x_t, a_t). \quad (40)$$

A learned world model approximates

$$\hat{x}_{t+1} = F_\theta(\hat{x}_t, a_t). \quad (41)$$

Repeated application generates hypothetical futures:

$$\begin{aligned} \hat{x}_{t+1} &= F_\theta(x_t, a_t), \\ \hat{x}_{t+2} &= F_\theta(\hat{x}_{t+1}, a_{t+1}), \\ &\vdots \\ \hat{x}_{t+H} &= F_\theta(\hat{x}_{t+H-1}, a_{t+H-1}). \end{aligned}$$

The agent can compare alternative action sequences before acting in the real environment.

World model at gross level

A world model is a learned numerical model of a controlled dynamical system, packaged for state estimation, counterfactual prediction, planning, or policy learning.

13.2 The state is usually hidden

An agent typically observes

$$o_t = H(x_t)$$

rather than the complete state. A single image may reveal position but not velocity; a sensor may omit important environmental variables. The model therefore constructs a latent state from history:

$$z_t = E_\theta(o_0, a_0, o_1, a_1, \dots, o_t). \quad (42)$$

A typical latent world model contains

$$z_{t+1} \sim P_\theta(\cdot \mid z_t, a_t), \quad (43)$$

$$\hat{o}_t \sim D_\theta(\cdot \mid z_t), \quad (44)$$

$$\hat{r}_t = R_\theta(z_t, a_t), \quad (45)$$

where E_θ is an encoder or state estimator, P_θ is latent dynamics, D_θ is an observation decoder, and R_θ predicts reward, cost, danger, or another decision-relevant quantity.

The transition may be probabilistic because the environment is stochastic or because the latent state is incomplete.

13.3 What makes it a world model?

The term describes the *role* of the model more than one fixed architecture. A world model is expected to support some combination of

state estimation + action-conditioned prediction
+ counterfactual rollout + planning or policy learning

A passive predictor learns

$$o_t \mapsto o_{t+1}.$$

An interactive model learns

$$(o_t, a_t) \mapsto o_{t+1}$$

or the corresponding latent transition. The action input makes alternative futures available.

A world model can be implemented by a recurrent network, transformer, neural ODE, neural operator, diffusion or video model, state-space model, conventional simulator, or hybrid combination. The term says what the model does for an agent.

13.4 Planning with a world model

Suppose the latent dynamics and cost are

$$z_{t+1} = F_\theta(z_t, a_t), \quad c_t = C_\theta(z_t, a_t).$$

For a candidate action sequence $a_t, \dots, a_{t+H-1}$, the model predicts

$$\hat{J} = \sum_{k=0}^{H-1} \gamma^k C_\theta(\hat{z}_{t+k}, a_{t+k}). \quad (46)$$

A model-predictive controller chooses

$$(a_t^*, \dots, a_{t+H-1}^*) \in \arg \min \hat{J}, \quad (47)$$

executes the first action, observes the real world again, updates the latent state, and replans.

Alternatively, imagined rollouts train a policy $\pi_\phi(a | z)$ and a value function. The Dreamer family follows this broad pattern, learning latent dynamics and improving behavior through imagined trajectories (Hafner et al., 2023).

The early *World Models* work learned a compressed visual representation and recurrent dynamics, then trained a compact controller partly inside generated trajectories (Ha and Schmidhuber, 2018).

13.5 A world model need not reconstruct everything

A model used for decision-making need not reproduce every pixel or microscopic detail. It may retain only information needed for reward prediction, value estimation, or planning.

MuZero is a central example. It learns a representation, dynamics, and prediction model that recursively produces rewards, policy information, and values useful for tree search without reconstructing the full observed environment or being given the environment rules (Schrittwieser et al., 2020).

This is analogous to closure modeling. A closure does not reproduce every molecular trajectory; it preserves the unresolved influence needed by the coarse equations. A task-oriented world model need not reproduce the whole world; it needs a state and dynamics sufficient for the decisions under consideration.

13.6 Generative interactive environments

The phrase “world model” has expanded to include action-conditioned generative simulators. A model may approximate

$$P_{\theta}(o_{t+1:t+H} \mid o_{\leq t}, a_{t:t+H-1}), \quad (48)$$

the distribution of future observations conditioned on history and proposed actions.

Genie combines a spatiotemporal tokenizer, an autoregressive dynamics model, and a learned latent-action model to generate controllable interactive visual environments from video data (Bruce et al., 2024). Such systems are much larger and less explicitly physical than a PDE time-step operator, but mathematically they remain approximations to action-conditioned transition operators or probability kernels.

13.7 A neural operator can function as a world model

Consider a controlled fluid

$$\partial_t u = \mathcal{N}(u, a), \quad (49)$$

where a is a boundary control, body force, valve setting, or other intervention. The exact finite-time flow operator is

$$\Phi_{\Delta t} : (u_t, a_t) \mapsto u_{t+\Delta t}. \quad (50)$$

A neural operator may learn

$$\Phi_{\theta, \Delta t}(u_t, a_t) \approx \Phi_{\Delta t}(u_t, a_t).$$

Used only for fast simulation, it is a neural time-step operator. Used by an agent to compare alternative control sequences and choose actions, it is the dynamics component of a world model.

Thus the categories overlap:

A neural operator can be the transition model inside a world model.

Likewise, a Christlieb-style closure plus a moment solver and an observation model could form a world model for a controlled kinetic system.

13.8 World models and closure models

The analogy is deeper than terminology.

	Closure model	World model
Retained representation	Coarse variables or moments	Latent state or belief state
Discarded information	Fine scales, high moments, particles	Unobserved world state and irrelevant detail
Learned object	Missing effect on retained equations	Transition and consequence model
Purpose	Predict coarse dynamics	Predict action-conditioned futures and support decisions
Exact complication	Memory and stochastic orthogonal dynamics	Partial observability, memory, uncertainty, and policy-dependent data
Typical simplification	Local deterministic constitutive law	Markovian latent transition

Both seek a representation that is smaller than the full underlying system but sufficient for a specified predictive task.

13.9 The controlled transition-operator viewpoint

From a numerical-analysis perspective, the fundamental object is often

$$\mathcal{P}_{\Delta t} : (z, a) \mapsto \text{distribution of } z'. \quad (51)$$

For deterministic dynamics this reduces to

$$z' = F_{\Delta t}(z, a).$$

For stochastic or partially observed systems, the correct object is a Markov kernel

$$P(\mathrm{d}z' \mid z, a).$$

The world model approximates this kernel and composes it repeatedly.

This is closely related to system identification, state-space modeling, data assimilation, surrogate simulation, reduced-order modeling, and model-predictive control. The distinctive AI framing emphasizes that the learned model is used internally by an agent to imagine and evaluate consequences.

13.10 Why world modeling is harder than ordinary supervised approximation

13.10.1 Compounding error

A small one-step error does not imply a small rollout error:

$$F_{\theta}(z, a) \approx F(z, a) \not\Rightarrow F_{\theta}^H(z) \approx F^H(z).$$

This is the learned analogue of consistency and stability in numerical time integration.

13.10.2 Distribution shift under planning

The model is trained on actions that were taken. A planner evaluates actions that may be very different. It can push the model into regions with little training support and may even exploit model errors rather than discover genuinely good actions.

13.10.3 Hidden state and memory

If z_t does not contain enough information, no transition

$$z_{t+1} = F(z_t, a_t)$$

can be exact. One may need recurrence, a richer belief state, a window of past observations, latent stochastic variables, or an explicit memory kernel.

13.10.4 Multiple possible futures

A deterministic average may be meaningless when futures branch. The model should represent

$$z_{t+1} \sim P_\theta(\cdot \mid z_t, a_t)$$

or a distribution over future observation sequences.

13.10.5 Task relevance versus physical fidelity

A task-oriented latent model can make good decisions while representing the physical world incompletely. Conversely, a visually convincing video model may fail to preserve the causal consequences of actions. Prediction quality must be judged relative to the model's intended use.

13.11 The grossest possible world-model summary

One-line definition

A world model is a learned numerical approximation of enough state, dynamics, observations, and action consequences to act as an internal simulator for an agent.

In the same notation used throughout this chapter,

$$\begin{aligned} \text{PINN: } & (x, t) \mapsto u(x, t), \\ \text{neural operator: } & \text{problem data} \mapsto \text{solution}, \\ \text{closure: } & \text{resolved variables} \mapsto \text{missing term}, \\ \text{world model: } & (\text{latent state, action}) \mapsto (\text{next latent state, predicted consequences}). \end{aligned} \tag{52}$$

The additional ingredient is not a fundamentally new kind of approximation. It is recursive, action-conditioned use of the approximation for imagination, planning, and control.

14 A single conceptual ladder

The methods discussed here can be organized by how much of the mathematical model the network replaces.

1. **Approximate one coefficient or constitutive scalar.**

$$\mu_{\text{eff}} = \mu_{\theta}(\text{local state}).$$

2. **Approximate one closure tensor or missing flux.**

$$\tau = \mathcal{C}_{\theta}(\bar{u}, \nabla \bar{u}, \dots).$$

3. **Approximate a reduced vector field.**

$$\dot{z} = F_{\theta}(z).$$

4. **Approximate a finite-time map.**

$$z_{n+1} = \Phi_{\theta}(z_n).$$

5. **Approximate a solution operator over a family.**

$$a(\cdot) \mapsto \mathcal{G}_{\theta}(a)(\cdot).$$

6. **Approximate a latent controlled simulator.**

$$(z_t, a_t) \mapsto P_{\theta}(\text{d}z_{t+1}, \text{d}r_t, \text{d}o_{t+1} \mid z_t, a_t).$$

Moving down this list generally increases the fraction of the simulator delegated to the learned model. It may increase speed and flexibility, but it also makes it harder to retain classical guarantees and to diagnose errors.

15 Practical questions for reading a scientific-ML paper

When confronted with a new “physics-informed,” “operator-learning,” “closure,” or “world-model” paper, the following questions usually reveal what is actually happening.

15.1 What is the target object?

Is the network approximating

$$u, \quad \mathcal{G}, \quad \mathcal{C}, \quad F, \quad \Phi, \quad h, \quad \text{or} \quad P(\text{d}z' \mid z, a)?$$

This is the first and most important classification.

15.2 Where does the supervision come from?

Possible answers include

- a strong-form differential-equation residual;
- a weak or variational identity;
- simulated input-output pairs;
- trajectories or state-transition pairs;
- high-fidelity closure targets;
- particle data;
- experimental observations;
- rewards, values, or planning targets.

15.3 What structure is hard, and what structure is soft?

Does the architecture guarantee conservation, hyperbolicity, positivity, symplecticity, invariance, or an attracting manifold? Or are these properties merely penalized on training samples?

15.4 What remains of the conventional solver?

Does the network replace the complete solve? Does it provide one flux inside a finite-volume method? Does an ODE solver integrate a learned right-hand side? Is a learned map simply iterated?

15.5 What is the relevant notion of generalization?

Generalization may mean a new initial condition, coefficient field, Knudsen number, forcing, geometry, action policy, time horizon, mesh, or physical regime. These are not interchangeable claims.

15.6 Is the model tested in closed loop?

For a closure, one wants an a posteriori PDE test. For a learned vector field, one wants long-time attractor and spectral tests. For a flow map, one wants recursive rollout. For a world model, one wants planning or policy performance under actions that may differ from the data-collection policy.

15.7 What failure is ruled out mathematically?

A useful guarantee is specific:

- eigenvalues remain real;
- covariance remains positive semidefinite;

- mass is conserved;
- a symplectic form is preserved;
- an invariant manifold exists;
- entropy production is nonnegative.

It should not be confused with a universal accuracy guarantee.

16 Final synthesis

Closure theory, PINNs, neural operators, structured learned dynamics, and world models belong to one broad landscape of numerical approximation, but they occupy different locations in a computational pipeline.

Closure theory

Closure begins when information is discarded. The reduced equations retain the effects of discarded scales through stresses, fluxes, correlations, memory, or stochastic terms. A closure supplies a tractable approximation to those effects.

PINNs

A classical PINN uses a neural network as a trial function for one solution and uses the governing equation primarily through a residual-based optimization problem. It normally requires a known governing equation, although that equation need not literally be a PDE.

Neural operators

A neural operator approximates a map between functions and amortizes a family of solves. It does not inherently require knowledge of a PDE; paired function data are enough. A PINO adds equation constraints to operator learning.

Christlieb

Christlieb's characteristic program learns missing kinetic closures inside retained moment or kinetic equations. The governing hierarchy identifies what is missing, while architecture and algebra enforce properties such as hyperbolicity, Galilean invariance, conservation, or entropy production.

Tang

Tang's work spans learned vector fields, attracting slow manifolds, symplectic maps, energetic variational laws, and neural operators. Its unifying feature is structure-preserving learning for reliable dynamics rather than one universal PINN recipe.

World models

A world model is an internal simulator for an agent. It approximates latent state evolution and action consequences, then uses recursive counterfactual rollouts for planning or policy learning. A neural operator, learned closure plus solver, or structured flow map can serve as a component of a world model.

At the highest level, all of these methods use neural networks as numerical approximation devices. The intellectually important distinctions are the identity of the approximated object, the source of supervision, the mathematical structure imposed on the approximation, and the way the learned object is composed with the rest of the model.

A Compact glossary

Closure

A model for unresolved terms that appear after averaging, filtering, taking moments, projecting modes, or otherwise discarding information.

Constitutive relation

A relation specifying material response, such as stress versus strain rate or heat flux versus temperature gradient. It is a form of closure.

RANS

Reynolds-averaged Navier–Stokes; a mean-flow model in which all turbulent fluctuations are represented through modeled stresses and fluxes.

LES

Large-eddy simulation; a filtered model that resolves large eddies and closes transfer across a selected spatial scale.

Moment closure

A relation expressing an unresolved high-order moment or its flux in terms of retained lower moments.

Mori–Zwanzig reduction

A projection formalism showing that exact reduced dynamics generally contain an instantaneous term, memory, and fluctuating or orthogonal dynamics.

PINN

In the narrow classical sense, a coordinate neural network trained as a solution ansatz using differential-equation residuals, initial and boundary conditions, and possibly observations.

Neural operator

A parameterized map between function spaces, usually trained to approximate a family of solution or transition operators.

PINO

A physics-informed neural operator that combines operator learning with equation-based constraints.

Neural ODE

A model in which a neural network parameterizes the right-hand side of an ODE and a numerical integrator constructs trajectories.

Flow map

A finite-time state-transition map $\Phi_{\Delta t} : z_t \mapsto z_{t+\Delta t}$.

Hard constraint

A property guaranteed for every model in the parameterized class, rather than encouraged by a finite penalty term.

World model

A learned representation of state, dynamics, observations, and action consequences used as an internal simulator for prediction, planning, or policy learning.

Latent state

An internal representation intended to summarize the observation history relevant to future prediction or control.

Transition kernel

A conditional probability law $P(dz' | z, a)$ for the next state given current state and action.

B One-page comparison table

Paradigm	Approximated object	Equation knowledge	What remains numerical?	Characteristic question
RANS/LES closure	Stress or flux	Known resolved PDE	Full reduced PDE solve	How do discarded scales act on resolved scales?
Kinetic moment closure	High moment or gradient relation	Known hierarchy	Closed moment PDE solve	How can a finite moment set remain predictive and well posed?
Classical PINN	One solution function	Residual normally known	Nonlinear optimization over network parameters	Can one neural trial function fit equation and observations?
Neural operator	Function-to-function map	Optional	Mostly one learned inference call	Can one model amortize a family of solves?
PINO	Function-to-function map	Known residual	Operator inference, possibly fine-tuning	Can operator learning combine data and equation supervision?

Paradigm	Approximated object	Equation knowledge	What remains numerical?	Characteristic question
Neural ODE	Vector field	Optional	ODE integration	Can trajectories identify a stable continuous-time law?
Symplectic map network	Finite-time map	No PDE required	Recursive map evaluation	Can long-time geometry be preserved exactly?
Fast-slow network	Coordinates, vector field, manifold	No PDE required	Full or reduced ODE integration	Can an attracting closure manifold be learned by design?
World model	Latent controlled transition kernel	No PDE required	Rollout, planning, policy optimization	Can an agent imagine useful action-conditioned futures?

References

- Jake Bruce, Michael D. Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie C. Y. Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, and Tim Harley. Genie: Generative interactive environments, 2024.
- J. W. Burby, Qi Tang, and Romit Maulik. Fast neural Poincaré maps for toroidal magnetic fields. *Plasma Physics and Controlled Fusion*, 63(2):024001, 2021.
- Yongsheng Chen, Wei Guo, Qi Tang, and Xinghui Zhong. Reduced-order modeling of hamiltonian dynamics based on symplectic neural networks, 2025.
- Andrew J. Christlieb, Mingchang Ding, Juntao Huang, and Nicholas A. Krupansky. Hyperbolic machine learning moment closures for the BGK equations. *Multiscale Modeling & Simulation*, 23(1):187–217, 2025.
- Yu Deng and Zaher Hani. Long time justification of wave turbulence theory, 2024. Version 2.
- Yu Deng, Zaher Hani, and Xiao Ma. Long time derivation of the Boltzmann equation from hard sphere dynamics, 2025a. Version 3.
- Yu Deng, Zaher Hani, and Xiao Ma. Hilbert’s sixth problem: Derivation of fluid equations via Boltzmann’s kinetic theory, 2025b.
- Valentin Duruisseaux, Joshua W. Burby, and Qi Tang. Approximation of nearly-periodic symplectic maps via structure-preserving neural networks, 2022.
- Massimo Germano, Ugo Piomelli, Parviz Moin, and William H. Cabot. A dynamic subgrid-scale eddy viscosity model. *Physics of Fluids A*, 3(7):1760–1765, 1991.

- David Ha and Jürgen Schmidhuber. World models, 2018.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2023.
- Juntao Huang, Yingda Cheng, Andrew J. Christlieb, and Luke F. Roberts. Machine learning moment closure models for the radiative transfer equation i: Directly learning a gradient-based closure, 2021a.
- Juntao Huang, Yingda Cheng, Andrew J. Christlieb, and Luke F. Roberts. Machine learning moment closure models for the radiative transfer equation iii: Enforcing hyperbolicity and physical characteristic speeds, 2021b.
- Juntao Huang, Yingda Cheng, Andrew J. Christlieb, Luke F. Roberts, and Wen-An Yong. Machine learning moment closure models for the radiative transfer equation ii: Enforcing global hyperbolicity in gradient-based closures, 2021c.
- Nikola B. Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- Robert H. Kraichnan. The structure of isotropic turbulence at very high reynolds numbers. *Journal of Fluid Mechanics*, 5(4):497–543, 1959.
- C. David Levermore. Moment closure hierarchies for kinetic theories. *Journal of Statistical Physics*, 83(5–6):1021–1065, 1996.
- Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021a.
- Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations, 2021b.
- Alec J. Linot, Joshua W. Burby, Qi Tang, Prasanna Balaprakash, Michael D. Graham, and Romit Maulik. Stabilized neural ordinary differential equations for long-time forecasting of dynamical systems. *Journal of Computational Physics*, 474:111838, 2023.
- Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- Yubin Lu, Xiaofan Li, Chun Liu, Qi Tang, and Yiwei Wang. Learning generalized diffusions using an energetic variational approach. *Communications in Computational Physics*, 40(2):389–412, 2026.
- Hazime Mori. Transport, collective motion, and brownian motion. *Progress of Theoretical Physics*, 33(3):423–455, 1965.
- Stephen B. Pope. *Turbulent Flows*. Cambridge University Press, 2000.

- Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588: 604–609, 2020.
- Daniel A. Serino, Allen Alvarez Loya, Joshua W. Burby, Ioannis G. Kevrekidis, and Qi Tang. Fast-slow neural networks for learning singularly perturbed dynamical systems. *Journal of Computational Physics*, 537:114090, 2025. doi: 10.1016/j.jcp.2025.114090.
- Joseph Smagorinsky. General circulation experiments with the primitive equations. i. the basic experiment. *Monthly Weather Review*, 91(3):99–164, 1963.
- Hao Wang, Chi-Wang Shu, and Qi Tang. LGNO: A local-global neural operator for hyperbolic conservation laws, 2026.
- Yue Zhao, Joshua W. Burby, Andrew Christlieb, and Huan Lei. Data-driven construction of a generalized kinetic collision operator from molecular dynamics, 2025.
- Robert Zwanzig. Memory effects in irreversible thermodynamics. *Physical Review*, 124(4):983–992, 1961.